

Lesson 6: Denial of Service (DoS)

Part 1: Goal and Vulnerability Summary

This lesson demonstrates a Denial of Service (DoS) weakness in the billing path of the DVSA serverless application.

Part 2: Root Cause

The billing path accepts repeated requests without rate limiting, allowing resource exhaustion.

Part 3: Environment and Setup

AWS Lambda, API Gateway, and CloudWatch are used in this serverless environment.

Part 4: Reproduction Steps

Send repeated parallel requests and observe failures and slowdown.

Part 5: Evidence

Terminal output shows failures and degraded performance under load.

Part 6: Fix Strategy

Apply rate limiting, throttling, and concurrency control.

Part 7: Code / Config Changes

API Gateway throttling and Lambda concurrency limits added.

Part 8: Verification

System remains stable under load after fix.

Part 9: Analysis

Requests overwhelm backend causing availability issues.

Part 10: Takeaway

Serverless systems still need protection against abuse.

Lambda > Functions > DVSA-ADMIN-GET-ORDERS

DVSA-ADMIN-GET-ORDERS

ThrottleCopy ARNActions

This function belongs to an application. [Click here](#) to manage it.

Function overview

DiagramTemplate

DVSA-ADMIN-GET-ORDERS

Layers (0)

+ Add trigger

+ Add destination

Related functions

Select a function

Function ARN

arn:aws:lambda:us-east-1:883425316372:func:DVSA-ADMIN-GET-ORDERS

Description

-

Last modified

23 hours ago

Application

serverlessrepo-OWASP-DVSA

Export to Infrastructure Composer

Download

Code

Test

Monitor

Configuration

Aliases

Versions

Code source

Open in Visual Studio Code

Upload from

(

Info

Tutorials

Learn how to implement common use cases in AWS Lambda.

Create a simple web app

In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
- Invoke your function through its function URL

[Learn more](#)

Start tutorial

Lambda > Functions > DVSA-ORDER-MANAGER

DVSA-ORDER-MANAGER

Export to Infrastructure ComposerDownload

Successfully updated the function "DVSA-ORDER-MANAGER".

Function overview

DiagramTemplate

DVSA-ORDER-MANAGER

Layers (0)

+ Add destination

Related functions

Select a function

Function ARN

arn:aws:lambda:us-east-1:883425316372:func:DVSA-ORDER-MANAGER

Description

-

Last modified

23 hours ago

Application

serverlessrepo-OWASP-DVSA

API Gateway

+ Add trigger

Export to Infrastructure Composer

Download

Code

Test

Monitor

Configuration

Aliases

Versions

Code source

Open in Visual Studio Code

Upload from

VS Code Explorer

JS order-manager.js

JS order-manager.js > handler > handler

98 exports.handler = async (event)

343 const lambdaResponse =

Info

Tutorials

Learn how to implement common use cases in AWS Lambda.

Create a simple web app

In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
- Invoke your function through its function URL

[Learn more](#)

Start tutorial

← → DVSA-ORDER-MANAGER

EXPLORER

DVSA-ORDER-MANAGER

node_modules

base64url

buffer

es6-promise

ieee754

lodash

long

node-forge

node-jose

node-serialize

pako

process

uuid

JS order-manager.js

package.json

DEPLOY Undeployed

Deploy (⌘⌘U)

Test (⌘⌘I)

TEST EVENTS [NONE SELECTED]

Create new test event

JS order-manager.js X

JS order-manager.js > handler > handler

```
1  const serialize = require('node-serialize');
2  const { LambdaClient, InvokeCommand } = require("@aws-sdk/client-lambda");
3  const { CognitoIdentityProviderClient, AdminGetUserCommand } = require("@aws-sdk/client-cognito-identity-provider");
4  const jose = require('node-jose');
5
6
7
8  exports.handler = (event, context, callback) => {
9      // console.log(JSON.stringify(event));
10     var req = serialize.serialize(event.body);
11     var headers = serialize.unserialize(event.headers);
12     var auth_header = headers.Authorization || headers.authorization;
13     var token_sections = auth_header.split('.');
14     var auth_data = jose.util.base64url.decode(token_sections[1]);
15     var token = JSON.parse(auth_data);
16     var user = token.username;
17     var isAdmin = false;
18
19     var params = {
20         UserPoolId: process.env.userpoolid,
21         Username: user
22     };
23
24     try {
25         const cognitoIdentityServiceProvider = new CognitoIdentityProviderClient();
26         const command = new AdminGetUserCommand(params);
27         const userData = cognitoIdentityServiceProvider.send(command);
28
29         userData.then((userData) => {
30             // console.log("userData", JSON.stringify(userData));
31             var len = Object.keys(userData.UserAttributes).length;
32             for (var i=0; i< len; i++) {
33                 if (userData.UserAttributes[i].Name === "custom:is_admin") {
34                     isAdmin = userData.UserAttributes[i].Value;
```

Enter Shipping Details

Name

Non obcaecat volupt

Address

Natus voluptas id ip

Email (you will receive a receipt with a link 📧)

Email

Phone

Adipisicing numquam

Submit

Confirmation for order: 0f79e107-6219-43c8-809e-47cc3171e585



Rampage
Nintendo

Total Price: \$ 40

Confirmation Token: XQd4wvsj7ZJj

Sending 100 parallel requests to /billing endpoint..

Request 1: 200 OK
Request 2: 200 OK
Request 3: 200 OK
Request 25: 503 Service Unavailable
Request 30: Timeout
Request 45: 429 Too Many Requests
Request 60: Failed - Connection reset
Request 75: 500 Internal Server Error

Summary:
Success: 30
Failed: 70
Average response time: 5.8s